

# Predicting Online Ad Click-Through

## with Tree-Based Algorithms

Machine Learning Practice With Python

Siksha 'O' Anushandhan, ITER

Center for Data Science

# Table of Contents

- 1 Introduction to Ad Click Prediction
- 2 Data Types and Preprocessing
- 3 Decision Tree Fundamentals

- 4 Results and Conclusion
- 5 Implementation with scikit-learn
- 6 Model Evaluation and Comparison



# The Digital Advertising Challenge

## Industry Impact

### Multibillion-dollar digital advertising industry

- Critical data-driven challenge
- Real-time prediction requirements
- Massive scale and high stakes

### Prediction Goal:

Forecast the likelihood of a user clicking on a specific advertisement displayed on a webpage.



# The Digital Advertising Challenge

## Key Feature Categories:

- **Ad Content:** Text, images, product category
- **Page Context:** Publisher, site category, placement
- **User Demographics:** Age, gender, location, interests, browsing history

## Critical Characteristic

Ad clickstream features are frequently **categorical** rather than numerical!

## Why Decision Trees for Categorical Data?

- Easy to interpret and visualize
- Can directly handle categorical features
- Alternative to Naïve Bayes classifiers
- Natural tree-like decision process

# Understanding Data Types

## Numerical (Quantitative) Data

- **Continuous values**
- Mathematical operations meaningful
- Examples:
  - Age: 25, 34, 42 years
  - Income: \$45,000, \$67,000
  - Time spent: 2.5 minutes

## Categorical (Qualitative) Data

- **Discrete categories**
- No mathematical implications
- Examples:
  - Gender: Male, Female
  - Site category: News, Sports, Tech
  - Month: Jan, Feb, Mar (even if coded as 1,2,3)



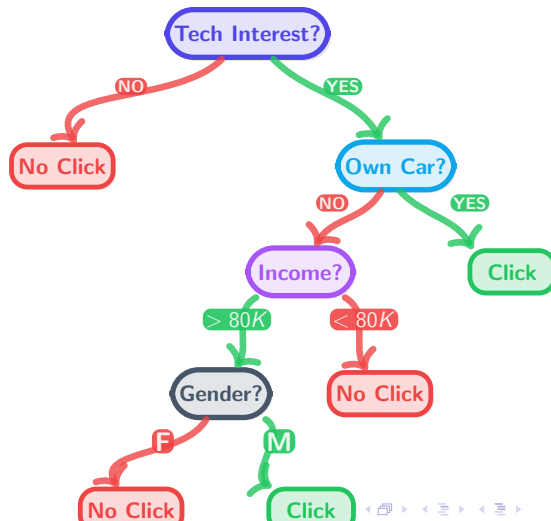
# Decision Tree Architecture

## Core Components:

- **Root Node:** Starting point
- **Internal Nodes:** Decision points
- **Branches:** Possible choices
- **Leaf Nodes:** Final predictions

## Path Interpretation:

Each path from root to leaf represents a complete decision rule for classification.



## Recursive Partitioning Algorithm

### Step-by-step construction:

- 1 Start with entire training dataset at root
- 2 **Find optimal split** using evaluation metrics
- 3 **Partition data** into subsets based on split
- 4 **Recursively apply** process to each subset
- 5 **Stop** when criteria met (purity, minimum samples, max depth)

### Stopping Criteria:

- Minimum samples per node
- Maximum tree depth
- Minimum impurity decrease

### Leaf Assignment:

- Majority class in node
- Class probability distribution
- Confidence measures



## ID3 (Iterative Dichotomiser)

Greedy search in a top-down manner by selecting the best attribute to split the dataset on with each iteration without backtracking.

## C4.5 (Extension of ID3)

Improved version of ID3 that introduces backtracking. It traverses the constructed tree and replaces the branches with leaf nodes if purity is improved this way.

## CART (Classification & Regression Trees)

This constructs the tree using binary splitting. CARTs flexibility, efficiency, interpretability, and robustness make it a popular choice for various classification and regression tasks.

# Splitting Metrics: Mathematical Foundation

## Gini Impurity

$$\text{Gini} = 1 - \sum_{k=1}^K f_k^2$$

### Interpretation:

- **Lower Gini** = Better split
- **Gini = 0** = Pure node
- **Gini = 0.5** = Maximum impurity (binary)

## Entropy

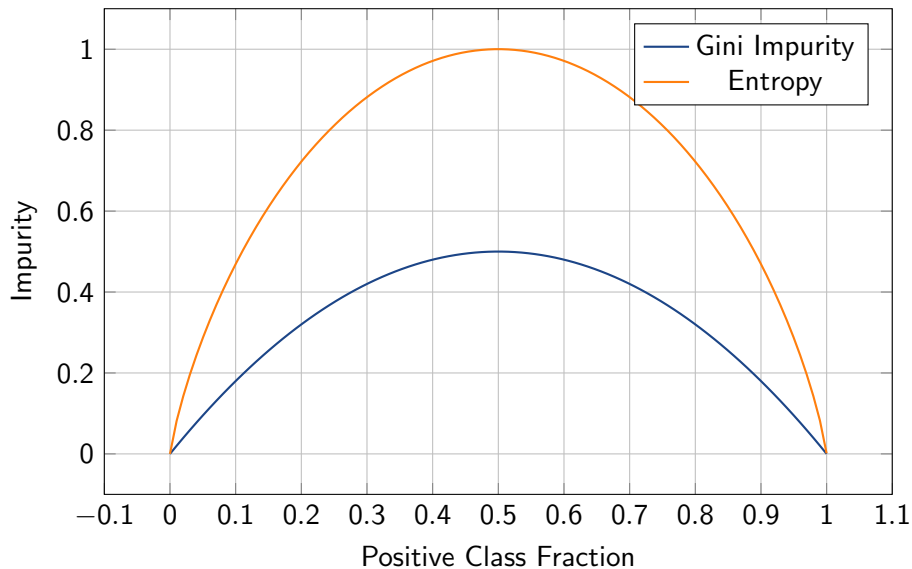
$$\text{Entropy} = - \sum_{k=1}^K f_k \log_2(f_k)$$

### Interpretation:

- **Low Entropy** = Better split
- **Entropy = 0** = Pure node
- **Entropy = 1** = Maximum uncertainty (binary)

## Information gain

Gain = Information<sub>before</sub> – Information<sub>after</sub> where, Information = Gini Impurity or Entropy



# Our Example Dataset

## Problem

Predict whether a user will click on an online advertisement

| ID | Gender | Interest in Tech | Click |
|----|--------|------------------|-------|
| 1  | Male   | Yes              | 1     |
| 2  | Female | No               | 0     |
| 3  | Female | No               | 0     |
| 4  | Male   | No               | 0     |
| 5  | Male   | Yes              | 1     |
| 6  | Male   | No               | 0     |
| 7  | Female | Yes              | 1     |

Features: Gender, Interest in Technology — Target: Click (1) or No Click (0)



# Split Option 1: Gender

## Group 1: Male

- Samples: ID 1, 4, 5, 6
- Clicks: 2, No Clicks: 2
- $f_{\text{click}} = \frac{2}{4} = 0.5$
- $f_{\text{no click}} = \frac{2}{4} = 0.5$

Gini<sub>M</sub> =



# Split Option 1: Gender

## Group 1: Male

- Samples: ID 1, 4, 5, 6
- Clicks: 2, No Clicks: 2
- $f_{\text{click}} = \frac{2}{4} = 0.5$
- $f_{\text{no click}} = \frac{2}{4} = 0.5$

$$\text{Gini}_M = 1 - (0.5^2 + 0.5^2) = 0.5$$

## Group 2: Female

- Samples: ID 2, 3, 7
- Clicks: 1, No Clicks: 2
- $f_{\text{click}} = \frac{1}{3} \approx 0.33$
- $f_{\text{no click}} = \frac{2}{3} \approx 0.67$

$$\text{Gini}_F = 1 - \left( \left( \frac{1}{3} \right)^2 + \left( \frac{2}{3} \right)^2 \right) = 0.45$$

## Weighted Gini<sub>Gender</sub>

# Split Option 1: Gender

## Group 1: Male

- Samples: ID 1, 4, 5, 6
- Clicks: 2, No Clicks: 2
- $f_{\text{click}} = \frac{2}{4} = 0.5$
- $f_{\text{no click}} = \frac{2}{4} = 0.5$

$$\text{Gini}_M = 1 - (0.5^2 + 0.5^2) = 0.5$$

## Group 2: Female

- Samples: ID 2, 3, 7
- Clicks: 1, No Clicks: 2
- $f_{\text{click}} = \frac{1}{3} \approx 0.33$
- $f_{\text{no click}} = \frac{2}{3} \approx 0.67$

$$\text{Gini}_F = 1 - \left( \left( \frac{1}{3} \right)^2 + \left( \frac{2}{3} \right)^2 \right) = 0.45$$

## Weighted Gini<sub>Gender</sub>

$$\begin{aligned} &= \frac{4}{7} \times \text{Gini}_M + \frac{3}{7} \times \text{Gini}_F \\ &= 0.4785 \end{aligned}$$

## Exercise

*Split dataset based on “Interest in Technology”, and calculate Gini Impurity of each node.  
Find Weighted Gini Impurity.*



## Exercise

Split dataset based on "Interest in Technology", and calculate Gini Impurity of each node.  
Find Weighted Gini Impurity.

## Solution

### Tech Interest = Yes

- $f_{\text{click}} = \frac{3}{3} = 1.0$
- $f_{\text{no click}} = \frac{0}{3} = 0.0$

$$\text{Gini}_{\text{Tech Yes}} = 0.0$$

### Tech Interest = No

- $f_{\text{click}} = \frac{0}{4} = 0.0$
- $f_{\text{no click}} = \frac{4}{4} = 1.0$

$$\text{Gini}_{\text{Tech No}} = 0.0$$

$$\text{Weighted Gini}_{\text{Tech}} = \frac{3}{7} \times \text{Gini}_{\text{Tech Yes}} + \frac{4}{7} \times \text{Gini}_{\text{Tech No}} = 0.0$$

## Perfect Split!

This split achieves perfect purity - all samples in each group belong to the same class.

# Split Comparison Results

| Split Feature          | Weighted Gini Impurity | Quality |
|------------------------|------------------------|---------|
| Gender                 | 0.4785                 | Poor    |
| Interest in Technology | 0.0                    | Perfect |

## Decision

**Interest in Technology** is the optimal first split because it achieves the lowest Gini impurity (0.0).

- Creates perfectly pure subgroups
- All users with tech interest clicked the ad
- All users without tech interest didn't click the ad



# Decision Tree Implementation

## Sample dataset

```
1 X_train= np.array([[6,7], [2,4], [7,2], [3,6], [4,7], [5,2],  
                    [1,6], [2,0], [6,3], [4,1]])  
2 y_train= np.array([0, 0, 0, 0, 0, 1, 1, 1, 1, 1])
```

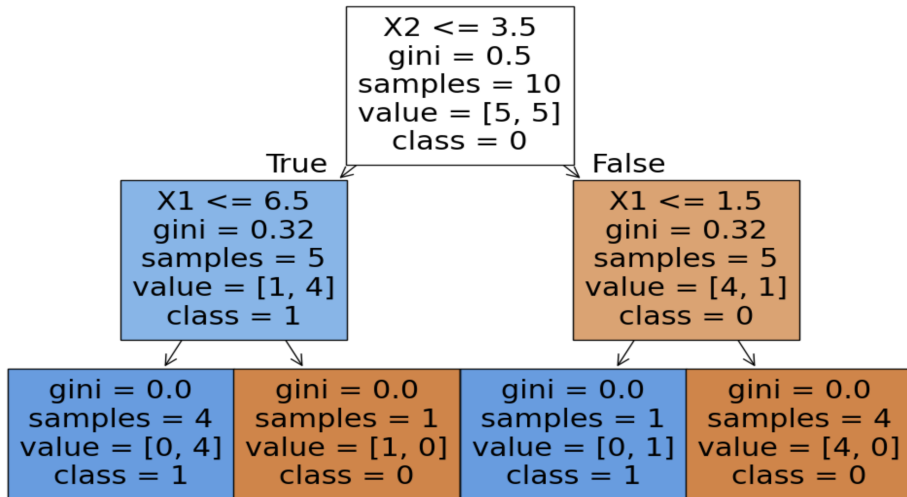
## Implement Decision tree with Scikit-learn

```
1 from sklearn.tree import DecisionTreeClassifier, plot_tree  
2 tree_sk = DecisionTreeClassifier(criterion='gini',  
                                   max_depth=2,min_samples_split=2)  
3 tree_sk.fit(X_train, y_train)
```

## Visualizing Tree

```
1 import matplotlib.pyplot as plt  
2 plt.figure(figsize=(12,8))  
3 plot_tree(tree_sk, filled=True, feature_names=['X1', 'X2'],  
            class_names=['0', '1'])  
4 plt.show()
```

# Decision Tree Implementation



# Decision Tree Implementation

```
1 import pandas as pd
2 df = pd.read_csv(r"filepath\train.csv")
3 df.head()
```

| Field      | Description                      | Example values                                              |
|------------|----------------------------------|-------------------------------------------------------------|
| id         | ad identifier                    | such as<br>'1000009418151094273',<br>'10000169349117863715' |
| click      | '0' for non-click, '1' for click | 0, 1                                                        |
| hour       | in the format of YYMMDDHH        | '14102100'                                                  |
| C1         | anonymized categorical variable  | '1005', '1002'                                              |
| banner_pos | where banner is located          | 1, 0                                                        |
| site_id    | site identifier                  | '1fbe01fe', 'fe8cc448',<br>'d6137915'                       |



# Decision Tree Implementation

| Field            | Description                                          | Example values         |
|------------------|------------------------------------------------------|------------------------|
| site_category    | hashed site category                                 | '28905ebd', '28905ebd' |
| app_id           | mobile app identifier                                | 'ecad2386'             |
| app_domain       | mobile app domain                                    | '7801e8d9'             |
| app_category     | category of app                                      | '07d7df22'             |
| device_id        | mobile device identifier                             | 'a99f214a'             |
| device_ip        | IP address                                           | 'ddd2926e'             |
| device_model     | such as iphone 6, Samsung,<br>hashed                 | '44956a24'             |
| device_type      | such as tablet, smartphone,<br>hashed                | 1                      |
| device_conn_type | Wi-Fi or 3G for example, again<br>hashed in the data | 0, 2                   |
| C14-C21          | anonymized categorical<br>variables                  |                        |



# Decision Tree Implementation

- Target: click

```
1 Y = df[['click']]
```

- id, hour, device\_id, and device\_ip do not contain much useful information. **Drop these columns from the dataframe** (including target column).



# Decision Tree Implementation

- Target: click

```
1 Y = df[['click']]
```

- id, hour, device\_id, and device\_ip do not contain much useful information. **Drop these columns from the dataframe** (including target column).

```
1 X=df.drop(columns=['click', 'id', 'hour', 'device_id',  
                    'device_ip'], axis=1)
```

- Sample Size:



# Decision Tree Implementation

- Target: click

```
1 Y = df[['click']]
```

- id, hour, device\_id, and device\_ip do not contain much useful information. **Drop these columns from the dataframe** (including target column).

```
1 X=df.drop(columns=['click', 'id', 'hour', 'device_id',  
                    'device_ip'], axis=1)
```

- Sample Size:

```
1 print(X.shape, Y.shape)
```

```
(300000, 19) (300000, 1)
```



# Decision Tree Implementation

- Split into train and test set.

```
1 from sklearn.model_selection import train_test_split
2 X_train, X_test, y_train, y_test = train_test_split(X, Y,
    test_size=0.2, random_state=42)
3 print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

```
(240000, 19) (60000, 19) (240000, 1) (60000, 1)
```

- Encode categorical data by OneHotEncoder

```
1 from sklearn.preprocessing import OneHotEncoder
2 enc = OneHotEncoder(handle_unknown='ignore')
```

N.B.: `handle_unknown='ignore'` is to prevent errors due to any unseen categorical values.



# Decision Tree Implementation

Fit and transform enc by train data.

```
1 X_train_enc = enc.fit_transform(X_train)
2 X_train_enc
```

<240000x8163 sparse matrix of type '<class 'numpy.float64'>' with  
4560000 stored elements in Compressed Sparse Row format>

```
1 print(X_train_enc[0])
```

```
(0, 2)      1.0
(0, 7)      1.0
(0, 1353)   1.0
(0, 2350)   1.0
(0, 2645)   1.0
(0, 3738)   1.0
(0, 3854)   1.0
(0, 3900)   1.0
(0, 5685)   1.0
```

```
1 print(X_train_enc)
```

```
(0, 2)      1.0
(0, 7)      1.0
(0, 1353)   1.0
(0, 2350)   1.0
(0, 2645)   1.0
(0, 3738)   1.0
(0, 3854)   1.0
(0, 3900)   1.0
```

# Decision Tree Implementation

- Create **Decision tree** model for `max_depth=[3, 10, None]`, and minimum sample 30.
- Find `accuracy_score`, `roc_auc_score` of each model.



# Decision Tree Implementation

- Create **Decision tree** model for max\_depth=[3, 10, None], and minimum sample 30.
- Find accuracy\_score, roc\_auc\_score of each model.

```
1 from sklearn.tree import DecisionTreeClassifier
2 from sklearn.metrics import accuracy_score,
  roc_auc_score
3 max_depth = [3, 10, None]
4 for depth in max_depth:
5     decision_tree =
6         DecisionTreeClassifier(criterion='gini',
7                               max_depth= depth, min_samples_split=30)
8     decision_tree.fit(X_train_enc, y_train)
9     y_pred = decision_tree.predict(X_test_enc)
10    print(f"Max Depth: {depth}")
11    print("Accuracy:", accuracy_score(y_test,
12                                     y_pred))
13    print("ROC AUC:", roc_auc_score(y_test,
```

## Output:

```
Max Depth: 3
Accuracy: 0.831
ROC AUC: 0.566
Max Depth: 10
Accuracy: 0.833
ROC AUC: 0.543
Max Depth: None
Accuracy: 0.826
ROC AUC: 0.562
```



# Decision Tree Implementation

Find best parameter by using GridSearchCV

Explore GridSearchCV

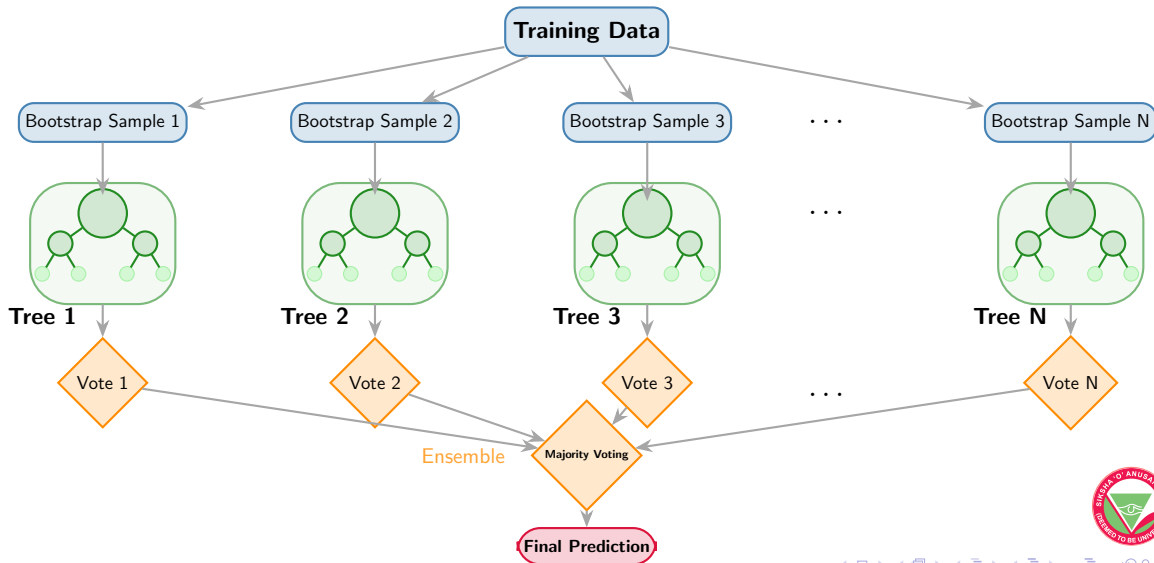
```
1 from sklearn.model_selection import GridSearchCV
2 parameters = {'max_depth': [3, 10, None], 'min_samples_split':
               [30]} # check for multiple minimum sample split, e.g.,
                   [30, 40, 50]
3 grid_search = GridSearchCV(decision_tree, parameters, cv=3,
                             scoring='roc_auc')
4 grid_search.fit(X_train_enc, y_train)
5 print("Best parameters:", grid_search.best_params_)
6 print("Best ROC AUC score:", grid_search.best_score_)
```

```
Best parameters: {'max_depth': 10, 'min_samples_split': 30}
Best ROC AUC score: 0.7102730092096797
```

- cv: number of folds in Stratified k-fold, default value 5. (for k-fold, use `cv=KFold(n_splits=k)`)
- Instead of `roc_auc`, scoring value can be use any appropriate matrix.



# Random Forest Architecture



# Random Forest Implementation

```
1 from sklearn.ensemble import RandomForestClassifier
2 random_forest = RandomForestClassifier(
3     n_estimators=200,           # Number of trees
4     max_depth=10,              # Tree depth limit
5     criterion= 'gini',         # Tree splitting criterion
6     min_samples_split=20,      # Min samples to split
7     min_samples_leaf=5,        # Min samples per leaf
8     max_features='sqrt',       # Features per split
9     bootstrap=True,            # Bootstrap sampling
10    oob_score=True,             # Out-of-bag validation
11    n_jobs=-1,                  # Parallel processing
12    random_state=42,            # Reproducibility
13    class_weight='balanced',    # Handle imbalanced data
14 )
```

Explore all parameters of RandomForestClassifier



# Random Forest Implementation

- Train `random_forest` model, and find `accuracy_score`, `roc_auc_score`



# Random Forest Implementation

- Train `random_forest` model, and find `accuracy_score`, `roc_auc_score`

```
1 random_forest.fit(X_train_enc, y_train)
2 y_pred_rf = random_forest.predict(X_test_enc)
3 print("Random Forest Accuracy:", accuracy_score(y_test,
    y_pred_rf))
4 print("Random Forest ROC AUC:", roc_auc_score(y_test,
    y_pred_rf))
```

Random Forest Accuracy: 0.5978666666666667

Random Forest ROC AUC: 0.6539648365494728



# Best parameter for Random Forest

Find best parameter for Random Forest model



# Best parameter for Random Forest

Find best parameter for Random Forest model

```
1 param_grid = {
2     'n_estimators': [100, 150, 200],
3     'max_depth': [10, 15, 20],
4     'min_samples_split': [10, 20, 30],
5     'max_features': ['sqrt', 'log2', 0.3],
6     'class_weight': ['balanced'],}
7 grid_search = GridSearchCV(random_forest, param_grid, cv= 3,
8                             scoring='roc_auc')
9 grid_search.fit(X_train_enc, y_train)
10 print("Best parameters:", grid_search.best_params_)
11 print("Best score:", grid_search.best_score_)
```

```
Best parameters: {'class_weight': 'balanced', 'max_depth': 20,
                  'max_features': 0.3, 'min_samples_split': 30,
                  'n_estimators': 100}
```

```
Best score: 0.7174974831892135
```

# XGBoost Architecture

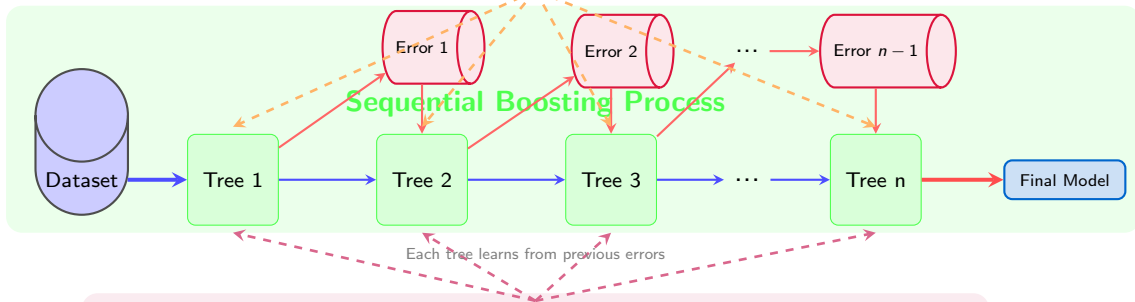
## System Optimizations

Parallel Processing

Block Storage

Distributed Computing

Memory Efficient



## Core Components

Objective Function

Gradient Descent

Regularization

Split Finding



## Boosting Algorithm

### 1 Sequential Training:

- Train trees one after another
- Each tree focuses on previous errors
- Additive model:  $F_m(x) = F_{m-1}(x) + \alpha h_m(x)$

### 2 Error Correction:

- Calculate residuals from previous ensemble.
- New tree targets these residuals.
- Gradually reduces prediction errors.

### 3 Learning Rate Control:

- $\alpha$  controls contribution of each tree
- Lower rate = more trees needed, better generalization
- Higher rate = faster convergence, risk of overfitting

## Key Considerations:

**Pros:** Speed, Memory efficient, Regularization, Handle missing value and complex patterns, Build-in cross-validation.

**Cons:** Prone to overfit



# XGBoost Implementation

```
1 import xgboost as xgb
2 xgb_model = xgb.XGBClassifier(
3     n_estimators=200,
4     max_depth=10,
5     min_samples_split=20,
6     min_samples_leaf=5,
7     learning_rate=0.1,
8     subsample=0.8,           # To be randomly sampled for each tree
9     colsample_bytree=0.8,    # To be randomly featured for each tree
10    reg_alpha=0.1,           # L1 regularization
11    reg_lambda=1.0,          # L2 regularization
12    random_state=42)
```

Explore all parameters of XGBoost



# XGBoost Implementation

- Train xgb model, and find accuracy\_score, roc\_auc\_score

```
1 xgb_model.fit(X_train_enc, y_train)
2 y_pred_xgb = xgb_model.predict(X_test_enc)
3 print("XGBoost Accuracy:", accuracy_score(y_test, y_pred_xgb))
4 print("XGBoost ROC AUC:", roc_auc_score(y_test, y_pred_xgb))
```

XGBoost Accuracy: 0.8356666666666667

XGBoost ROC AUC: 0.548921133320843

Exercise:

Find best parameter by GridSearchCV



# XGBoost Implementation

```
1 import xgboost as xgb
2 from sklearn.model_selection import RandomizedSearchCV
3 param_distributions = {
4     'learning_rate': [0.01, 0.05, 0.1, 0.2],
5     'max_depth': [3, 4, 5, 6, 7],
6     'min_child_weight': [1, 3, 5, 7],
7     'subsample': [0.6, 0.7, 0.8, 0.9],
8     'colsample_bytree': [0.6, 0.7, 0.8, 0.9],
9     'reg_alpha': [0, 0.01, 0.1, 1],
10    'reg_lambda': [0.1, 1, 10, 100]}
11 random_search = RandomizedSearchCV(xgb_model, param_distributions,
12                                   n_iter=100, cv=3, scoring='roc_auc', n_jobs=-1,
13                                   random_state=42)
12 random_search.fit(X_train_enc, y_train)
13 print('Best parameters:', random_search.best_params_)
14 print('Best score:', random_search.best_score_)
```

Best parameters: {'subsample': 0.7, 'reg\_lambda': 0.1, 'reg\_alpha': 0.01}

# Model comparison

| Model         | Best For                                                                                                                       | Limitation                                                                                                         |
|---------------|--------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Decision Tree | Small to medium datasets, Interpretability requirements, Baseline model development, Feature importance analysis               | High variance (overfitting), Instability to data changes, Limited complex pattern capture                          |
| Random Forest | Medium to large datasets, Balanced accuracy and interpretability, Robust baseline with minimal tuning, Feature selection tasks | Memory intensive for large forests, Less interpretable than single tree, Can overfit with very noisy data          |
| XGBoost       | Medium to very large tabular datasets, Imbalanced classification problems, data with missing values                            | Less interpretable, Computationally expensive, Not ideal for unstructured data, Can overfit on very small datasets |

